FundFlow — Project Master Brief & Architecture

AI-Powered Student Funding Disbursement & Assurance Platform | W3Node 2026

Executive Purpose: *This document gives our distributed 2-person development team full alignment on FundFlow's architecture, problem statement, real-world data, technical design, and Git workflow. Developer 1 (MERN Stack) and Developer 2 (Agentic Finance & XRPL) will use this manual alongside GitHub Copilot CLI to rapidly build and demo a working Agentic Finance MVP for the Ripple-sponsored W3Node Hackathon track.*

1. The Real-World Problem: Grounding in Verified Data

South African student funding (NSFAS, corporate bursaries, trust funds) suffers from recurring governance and disbursement failures. To win over hackathon judges and prove commercial viability, we cite these official investigative findings:

- **R5+ Billion in Ineligible Allocations (SIU):** The Special Investigating Unit (Proclamation R.88 of 2022) revealed over 40,000 ineligible beneficiaries inappropriately received allowances—including ghost students, dropouts, deceased identities, and students exceeding income thresholds.
- **R2+ Billion in Trapped Unallocated Cash:** Poor system integration between funding bodies and institutions left billions stranded in university accounts for years. The SIU had to claw back over R1.7 Billion from universities (e.g. UJ and UFS) due to lack of real-time payment reconciliation.
- **The R47-Billion Direct Payment Scandal (Werksmans Report):** In 2023, four fintech providers (eZaga, Coinvest, Norraco, Tenet Technology) were awarded direct disbursement tenders. The independent Werksmans report and court judgments declared the tenders unlawful and unconstitutional, exposing deep conflicts of interest.
- **Predatory Fee Deductions on Impoverished Students:** The fintech middlemen charged students monthly bank fees (R12 to R29/month) plus ATM withdrawal fees, eroding monthly allowances intended for basic food and accommodation.
- **Repeated Qualified Audits by Auditor-General (AGSA):** AGSA repeatedly flags student funding bodies for absence of automated internal controls, broken audit trails, and reconciliation books that do not match ledger records.

2. The Ripple Agentic Finance Criteria

The workshop led by Maxime Dienger (Ripple) established explicit criteria for what wins this track ($2,500 prize + UDAX accelerator interview):

Criteria	Ripple Requirement	FundFlow Implementation
Core Question	*'Would your CFO sign off?'* Requires clear limits, auditability, and emergency stop.	Admin sets mandates, budgets, and beneficiary allowances with an instant pause switch.

No Chatbots	Autonomous agent action. NOT a chatbot asking a human to click 'Approve'.	Autonomous worker cycle evaluates schedules and dispatches payments without human intervention.
Out-of-Agent Policy	Limits enforced where the agent cannot alter or bypass them.	Independent Policy Engine intercepts requests and checks mandates before signing.
The Blocked Demo	*'A demo where the agent gets blocked beats one that only shows the happy path.'*	Demo shows: (1) Valid payment settles, (2) Duplicate/over-budget payment is blocked with audit code.
XRPL Settlement	Actual transaction on XRPL Testnet with immutable hash verification.	Node.js `xrpl.js` signs and submits payments; records `tesSUCCESS` hashes.

3. FundFlow System Architecture

FundFlow is built as a clean modular monolith using the MERN stack with dedicated security boundaries for AI and XRPL.

- **Frontend (React + Vite):** Split into (1) *Admin Dashboard* (mandates, limits, live audit stream, kill-switch) and (2) *Student Portal* (status cards, payment schedule, direct XRPL Explorer transaction links). Styled with a high-contrast Navy & Cyan fintech theme.
- **Backend REST API (Node.js & Express):** Houses CRUD controllers for programmes, beneficiaries, payment schedules, and audit records.
- **AI Disbursement Agent:** Autonomous service that evaluates active mandates and scheduled disbursements, prepares payment instructions, and submits them to the policy engine. *The agent never has access to private keys or wallet seeds!*
- **Independent Policy Engine (Zero-Trust Gate):** An independent, non-AI security module. Validates: (1) Is beneficiary whitelisted? (2) Does amount exceed student allowance? (3) Does program have sufficient budget? (4) Has this payment already occurred (Idempotency lock)? (5) Is programme active?
- **XRPL Transaction Service (`xrpl.js`):** Securely holds server-side signing keys. Submits transactions to XRPL Testnet, waits for validation, checks `tesSUCCESS`, and returns immutable transaction hashes to the audit log.
- **MongoDB / Mongoose:** Stores programmes, synthetic students, payment schedules, and immutable audit logs.

End-to-End Payment Sequence Flow:

1. Admin defines Funding Programme (Budget: R50,000, Whitelisted Students, Monthly Allowance: R1,650).
2. AI Disbursement Agent scans database for due scheduled payments.
3. Agent prepares payment proposal: { studentId: 'STU-001', amount: 1650, programmeId: 'PRG-01' }.
4. Independent Policy Engine independently queries database mandate (ignoring any agent claims).
5. Policy Engine verifies rules: beneficiary valid, budget sufficient, payment not duplicate.
6. If Approved: Policy Engine passes approved instruction to XRPL Transaction Service.
7. XRPL Service signs and submits payment transaction to XRPL Testnet.
8. Testnet settles in ~4 seconds; returns `tesSUCCESS` and 64-character hash.
9. System records verified transaction hash in MongoDB Audit Log.
10. Student Portal shows 'Verified' green badge with link to `testnet.xrpl.org`.
11. If Blocked: Policy Engine rejects immediately with reason code (e.g., `ERR_DUPLICATE_PAYMENT`), no funds move, and the exception is logged for the Admin.

4. Two-Developer Team Division of Responsibilities

Area	Developer 1 (MERN Application)	Developer 2 (Agentic Finance & XRPL)
Focus	User Experience, Dashboards & REST Endpoints	Blockchain Settlement, Policy Gate & Agent Logic
Key Tasks	• React + Vite Admin Dashboard • Student Portal with XRPL Explorer links • Express routes & Mongoose models • Navy/Cyan UI styling & status badges • Frontend API integration	• XRPL Testnet connection (`xrpl.js`) • Faucet auto-funding & transaction service • Independent Policy Engine (Zero-Trust checks) • Autonomous AI agent evaluation loop • Security test scripts (duplicate/over-budget)
Shared	Collaborate on `docs/API_SPEC.md` so UI and backend services integrate seamlessly without waiting.	Review each other's pull requests and participate in demo rehearsal.

5. Git Workflow & Remote Collaboration Strategy

Because Developer 1 and Developer 2 are working remotely using Git and GitHub Copilot CLI, strict branch hygiene prevents merge conflicts:

- **Main Branch (`main`):** Always working, deployable, and demo-ready. Never push code directly to `main`.
- **Development Integration Branch (`dev`):** Where feature branches are merged after testing.
- **Feature Branches:**
  - feature/dev1-admin-student-ui (Developer 1)
  - feature/dev1-api-routes (Developer 1)
  - feature/dev2-xrpl-service (Developer 2)
  - feature/dev2-policy-engine (Developer 2)
  - feature/dev2-disbursement-agent (Developer 2)
- **Git Secret Protection:** .env and wallet seed files must NEVER be tracked in Git. The .gitignore file is the first file committed.
- **GitHub Copilot CLI Best Practices:**
  - Give Copilot specific context: *'Using xrpl.js, submit an XRP payment from seed X to student address Y and await tesSUCCESS.'*
  - Keep functions small and modular so Copilot generates exact, unit-testable code.

6. Developer Education: XRPL & Agentic Finance Primer

1. Understanding XRPL (XRP Ledger):
- **Speed & Settlement:** XRPL does not use proof-of-work or smart contract auctions. It uses the Ripple Protocol Consensus Algorithm (RPCA), settling in 3–5 seconds with irreversible finality.
- **Base Reserve:** Every XRPL account requires 1 XRP minimum reserve to exist on ledger.
- **Drops:** 1 XRP = 1,000,000 drops. When sending transactions via xrpl.js, use xrpl.xrpToDrops('10').
- **Result Verification:** Always verify result.meta.TransactionResult === 'tesSUCCESS'.

2. Understanding Autonomous Agents:
- An agent is not a chatbot. It operates on a continuous loop: *Sense* (read pending payment schedules) → *Plan* (construct transaction parameters) → *Act* (invoke the Policy Engine).
- **The Golden Rule of Agentic Security:** *Never give an LLM direct access to wallet signing keys or allow it to validate its own rules.* The agent is an unprivileged proposer; the Policy Engine is the deterministic gatekeeper.

7. Implementation Checklist (Milestones 1 to 3)

Milestone	Key Deliverables	Status
M1: Monorepo & Git	Initialize git, .gitignore, scaffold server/ and client/, configure DB fallback, define API_SPEC.md.	**In Progress**
M2: Policy & XRPL	XRPL Testnet connection, faucet auto-funding, independent policy validation rules, duplicate prevention tests.	Queued
M3: Agent & Dashboards	Autonomous agent scheduler, Admin Dashboard with kill-switch, Student Portal with explorer links, end-to-end demo rehearsal.	Queued